

№	Описание	Пример кода
1. Создание классов	определение структуры и поведения объектов с помощью ключевого слова <code>class</code> в языке программирования	<pre>class MyClass: pass</pre>
2. Создание объектов	создание экземпляров класса, которые содержат данные и методы	<pre>my_object = MyClass()</pre>
3. Статический класс	класс, методы которого являются статическими и не связаны с конкретным экземпляром объекта	<pre>class StaticClass: @staticmethod def my_static_method(): pass</pre>
4. Статический метод	метод, который связан с классом, а не с конкретным объектом, и может быть вызван без создания экземпляра класса	<pre>class MyClass: @staticmethod def static_method(): pass</pre>
5. Аксессоры (getter, setter, deleter)	методы для чтения, записи и удаления значений свойств объекта	<pre>class MyClass: def __init__(self): self.__value = None def get_value(self): return self.__value def set_value(self, value): self.__value = value</pre>

		<pre>def delete_value(self): self.__value = None</pre>
6. Инкапсуляция	объединение данных и методов в классе для обеспечения безопасности и контроля доступа к данным	<pre>class MyClass: def __init__(self): self.__value = 5 def get_value(self): return self.__value def set_value(self, value): if value < 0 or value > 10: raise ValueError("Значение должно быть в диапазоне 0..10") else: self.__value = value</pre>
7. Наследование	создание новых классов на основе существующих с использованием ключевого слова extends	<pre>class BaseClass: def base_method(self): pass class DerivedClass(BaseCl ass): def derived_method(self): pass</pre>
8. Полиморфизм	способность объектов с одинаковой спецификацией иметь различную реализацию в зависимости от	<pre>class Animal: def speak(self): print("Говорит животное") class Dog(Animal): def speak(self):</pre>

	контекста	<pre> print("Гав-гав") class Cat(Animal): def speak(self): print("Мяу-мяу") animal1 = Animal() animal2 = Dog() animal3 = Cat() animal1.speak() animal2.speak() animal3.speak() </pre>
9. Интерфейс	набор абстрактных методов, которые должны быть реализованы в производных классах	<pre> from abc import ABC, abstractmethod class IShape(ABC): @abstractmethod def draw(self): """ Метод для отрисовки фигуры """ class Circle(IShape): def draw(self): print("Рисуем круг") # Создание экземпляра и его использование circle = Circle() circle.draw() </pre>
10. Множественное наследование	возможность наследования от нескольких базовых классов	<pre> class BaseClass1: def base_method1(self): pass class BaseClass2: def </pre>

		<pre> base_method2(self): pass class DerivedClass(BaseCl ass1, BaseClass2): def derived_method(self): pass </pre>
11. Примесь (mixin)	механизм добавления функциональности в существующий класс без его модификации или наследования	<pre> class Mixin: def mixin_method(self): pass class BaseClass(Mixin): pass class DerivedClass(BaseCl ass): pass </pre>
12. Перегрузка	возможность определения нескольких методов с одинаковым именем, но разными параметрами и возвращаемыми значениями	<pre> from functools import singledispatch from datetime import date, datetime, time @singledispatch def format(label): return label @format.register def _(label: date): return label.strftime("%Y-%m -%d") @format.register(date time) def _(label): return </pre>

		<pre>label.strftime("%Y-%m -%d %H:%M:%S") @format.register(time) def _(label): return label.strftime("%H:%M :%S")</pre>
--	--	---